

Python Syntax

Basic Operations

```
2 + 2      # Addition
4 - 2      # Subtraction
2 * 3      # Multiplication
6 / 2      # Division (float result)
7 // 2     # Integer division
7 % 3      # Modulo (remainder)
2 ** 3     # Exponentiation
```

Variables

```
var = 10          # Assign value
price = 900       # Integer
brand = 'Apple'   # String
is_billed = True  # Boolean
```

Type Conversion

```
int(4.5)          # To integer: 4
float(5)          # To float: 5.0
str(900)          # To string: '900'
bool(1)           # To boolean: True
type(var)         # Check type
```

Print

```
print("Hello World")      # Basic print
print(2+2, "Python")      # Multiple values
print("hello", "world", sep="-") # Custom separator
print(f"Price: ${price}") # F-string (3.6+)
```

Dictionaries

```
my_dict = {'brand': 'Apple', 'price': 900}
# Create dictionary

my_dict['brand']      # Access by key
my_dict['RAM'] = 4    # Add/modify pair
my_dict.get('color', 'N/A') # Get with default
my_dict.keys()       # Get all keys
my_dict.values()     # Get all values
my_dict.items()      # Get all pairs
```

Conditionals

```
if condition:
    # code block
elif another_condition:
    # code block
else:
    # code block
# Ternary operator
x = a if condition else b
```

String Operations

```
s = "Hello, World!"

# String methods
s.upper()          # "HELLO, WORLD!"
s.lower()          # "hello, world!"
s.strip()          # Remove whitespace
s.replace("World", "Python") # "Hello, Python!"
s.split(",")       # ["Hello", " World!"]
```

Lists

```
my_list = [1, 2, 3, 4]      # Create list
my_list[0]                  # First element
my_list[-1]                 # Last element
my_list[1:3]                # Slice (idx 1,2)
my_list.append(5)           # Add to end
my_list.pop()               # Remove last
my_list.pop(1)              # Remove at index
my_list[2] = 10             # Modify element
len(my_list)                # Get length
min(my_list)                # Min value
max(my_list)                # Max value
```

Tuples

```
my_tuple = (1, 2, 3)      # Create tuple
my_tuple[0]                # Access element
```

Functions - Definition

```
# Basic function
def my_function():
    print("Hello World")

# With parameters
def display_info(name, age):
    print(f"{name} is {age} years old")

# With return value
def add(a, b):
    return a + b

# Default parameters
def greet(name="User"):
    print(f"Hello, {name}")

# Lambda function
multiply = lambda x, y: x * y
```

Functions - Advanced Parameters

```
# *args (variable positional arguments)
def sum_all(*numbers):
    return sum(numbers)

# Example call
sum_all(1, 2, 3, 4)          # Use *args

# **kwargs (variable keyword arguments)
def print_info(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")

# Example call
print_info(name="Alice", age=25)
                             # Use **kwargs
```

For Loops

```
for i in range(5):           # 0 to 4
```

```
print(i)

for i in range(2, 6):        # 2 to 5
    print(i)

for i in range(5, 21, 5):    # 5, 10, 15, 20
    print(i)
```

Loop Iterations

```
for item in my_list:         # Iterate list
    print(item)

for k, v in my_dict.items(): # Dict iteration
    print(k, v)
```

While Loops

```
i = 0
while i < 5:
    print(i)
    i += 1

# break and continue
while True:
    if condition:
        break      # Exit loop
    if another_condition:
        continue   # Skip to next iteration
```

List Comprehensions

```
[x*2 for x in range(5)]
                                # [0, 2, 4, 6, 8]

[x for x in range(10) if x % 2 == 0]
                                # [0, 2, 4, 6, 8]

['Yes' if x > 5 else 'No' for x in range(10)]
                                # Conditional
```

Exception Handling

```
try:
    # Code that might raise an exception
    result = 10 / 0
except ZeroDivisionError:
    # Handle specific exception
    print("Cannot divide by zero")
except Exception as e:
    # Handle all other exceptions
    print(f"Error occurred: {e}")
else:
    # Runs if no exceptions occurred
    print("No exceptions")
finally:
    # Always runs, with or without exception
    print("Cleanup")
```

File Operations

```
# Reading a file
with open('file.txt', 'r') as f:
    content = f.read() # Read entire file
    # OR
    lines = f.readlines() # List of lines

# Writing to a file
with open('file.txt', 'w') as f:
    f.write('Hello World') # Write string

# Appending to a file
with open('file.txt', 'a') as f:
    f.write('New line')
```

Functions - Advanced Parameters

```
# *args (variable positional arguments)
def sum_all(*numbers):
    return sum(numbers)

# Example call
sum_all(1, 2, 3, 4) # Use *args
```

```
# **kwargs (variable keyword arguments)
def print_info(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")

# Example call
print_info(name="Alice", age=25)
# Use **kwargs
```

Function Calls

```
my_function() # Basic call

display_info("Alice", 25) # Positional args

display_info(name="Bob", age=30)
# Keyword args

greet() # Default param
```

String Operations

```
s = "Hello, World!"

# String methods
s.upper() # "HELLO, WORLD!"
s.lower() # "hello, world!"
s.strip() # Remove whitespace
s.replace("World", "Python") # "Hello, Python!"
s.split(",") # ["Hello", " World!"]

# String formatting
name = "Alice"
age = 30
f"{name} is {age} years old" # f-string
"{ } is { } years old".format(name, age)
```

Modules and Imports

```
# Basic import
import math
```

```
math.sqrt(16)                # 4.0

# Import specific function
from math import sqrt
sqrt(16)                      # 4.0

# Import with alias
import numpy as np
np.array([1, 2, 3])

# Import all (not recommended)
from math import *
```

Classes and Objects

```
# Define a class
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def greet(self):
        return f"Hello, my name is {self.name}"

# Create an object
p = Person("Alice", 30)
p.name                # "Alice"
p.greet()              # "Hello, my name is Alice"
```

Working with Dates

```
from datetime import datetime, timedelta

# Current date and time
now = datetime.now()

# Specific date
date = datetime(2023, 12, 31)

# Date arithmetic
tomorrow = now + timedelta(days=1)
next_week = now + timedelta(weeks=1)
```

```
# Formatting
now.strftime("%Y-%m-%d")    # "2023-12-31"
```